

AI LAB EXAM-3

NAME:P.VARSHITHA

HTNO:2403A510G8

SET E-13

Batch:06 TASK1:

PROMPT 1: *In the Entertainment sector, a company is facing a problem managing and recommending movies, songs, or shows to users efficiently. Source code using data structures to store and process entertainment data*

CODE:

```
# ...existing code...

Entertainment sector example: data structures + simple AI-like recommender.
- Builds an inverted index and a Trie for fast search.
- Uses Jaccard similarity between movies (based on genres+tags) as a lightweight "AI"
- Aggregates scores from a user's rated items to produce recommendations using a heap

from collections import defaultdict, deque
import heapq
import math

# Simple movie dataset (expandable)
MOVIES = {
    1: {"title": "Space Quest", "genres": {"sci-fi", "adventure"}, "tags": {"space", "sci-fi"}, "rating": 4.5},
    2: {"title": "Romantic Days", "genres": {"romance", "drama"}, "tags": {"love", "romance"}, "rating": 3.8},
    3: {"title": "Alien Intrusion", "genres": {"sci-fi", "thriller"}, "tags": {"alien", "sci-fi"}, "rating": 4.2},
    4: {"title": "Comedy Nights", "genres": {"comedy"}, "tags": {"standup", "funny"}, "rating": 3.5},
    5: {"title": "Space Romance", "genres": {"sci-fi", "romance"}, "tags": {"space", "romance"}, "rating": 4.0},
    6: {"title": "Historic Battles", "genres": {"history", "action"}, "tags": {"war", "history"}, "rating": 4.1},
    7: {"title": "The Explorer", "genres": {"adventure"}, "tags": {"exploration", "adventure"}, "rating": 3.9},
    8: {"title": "Sci-Fi Laughs", "genres": {"sci-fi", "comedy"}, "tags": {"space", "comedy"}, "rating": 3.7}
}

# Trie for title prefix search
class TrieNode:
    def __init__(self):
        self.children = {}
        self.ids = set()
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, title, movie_id):
        node = self.root
```

```

def insert(self, title, movie_id):
    for ch in title.lower():
        if ch not in node.children:
            node.children[ch] = TrieNode()
            node = node.children[ch]
        node.ids.add(movie_id)
    node.is_end = True

def search_prefix(self, prefix):
    node = self.root
    for ch in prefix.lower():
        if ch not in node.children:
            return set()
        node = node.children[ch]
    return node.ids

# Build inverted index (tag/genre -> movie ids) and trie
def build_indexes(movies):
    inv = defaultdict(set)
    trie = Trie()
    for mid, meta in movies.items():
        trie.insert(meta["title"], mid)
        for g in meta["genres"]:
            inv[g].add(mid)
        for t in meta["tags"]:
            inv[t].add(mid)
    return inv, trie

# Jaccard similarity on the set of features (genres + tags)
def jaccard(a, b):
    if not a and not b:
        return 0.0
    inter = len(a & b)
    union = len(a | b)
    return inter / union if union else 0.0

```

```

# Precompute movie feature sets
def movie_features(movies):
    features = {}
    for mid, meta in movies.items():
        features[mid] = set(meta["genres"]) | set(meta["tags"])
    return features

# Recommend: for each unrated movie, score = sum(rating * similarity)
def recommend(movies, user_ratings, top_n=3):
    features = movie_features(movies)
    scored = []
    for mid in movies:
        if mid in user_ratings:
            continue
        score = 0.0
        for rated_mid, rating in user_ratings.items():
            sim = jaccard(features[mid], features[rated_mid])
            score += rating * sim
        if score > 0:
            heapq.heappush(scored, (-score, mid)) # max-heap via negation
    results = []
    for _ in range(min(top_n, len(scored))):
        s, mid = heapq.heappop(scored)
        results.append((mid, -s))
    return results

# Example usage & sample output
def main():
    inv_index, trie = build_indexes(MOVIES)

    # Example user ratings (movie_id -> rating 1..5)
    user_ratings = {
        1: 5.0, # loved Space Quest
        3: 3.0, # liked Alien Intrusion
    }

    print("User ratings:")

```

```

print("User ratings:")
for mid, r in user_ratings.items():
    print(f" {mid}: {MOVIES[mid]['title']} -> {r}")

recs = recommend(MOVIES, user_ratings, top_n=4)
print("\nTop recommendations (movie id, score):")
for mid, score in recs:
    print(f" {mid}: {MOVIES[mid]['title']} (score={score:.3f})")

# Demonstrate trie search and inverted index usage
prefix = "Space"
matched = trie.search_prefix(prefix)
print(f"\nTitle search prefix='{prefix}':")
for mid in matched:
    print(f" {mid}: {MOVIES[mid]['title']}")

tag = "space"
print(f"\nMovies with tag='{tag}' (from inverted index):")
for mid in sorted(inv_index[tag]):
    print(f" {mid}: {MOVIES[mid]['title']}")

if __name__ == "__main__":
    main()
...existing code...

```

OUTPUT:

```

User ratings:
  1: Space Quest -> 5.0
  3: Alien Intrusion -> 3.0

Top recommendations (movie id, score):
  5: Space Romance (score=1.857)
  8: Sci-Fi Laughs (score=1.857)
  7: The Explorer (score=1.667)

Title search prefix='Space':
  1: Space Quest
  5: Space Romance

Movies with tag='space' (from inverted index):
  1: Space Quest
  5: Space Romance
  8: Sci-Fi Laughs

```

OBSERVATION:

1. The program is a movie recommendation system using AI-like similarity (Jaccard similarity) to suggest movies based on genres and tags.
2. Data structures used include Trie (for title search), inverted index (tag/genre lookup), heap (top recommendations), and dictionary/set (storing movie info).
3. It computes recommendation scores using user ratings and similarity to other movies.
4. Supports fast search by prefix (Trie) and by tag/genre (inverted index).
5. AI assistance can help design efficient data structures, refactor code, and implement recommendation logic clearly and modularly.

TASK-2:

PROMPT 2: "You are an AI software engineering assistant. A transportation company has an old, unoptimized codebase that manages vehicle scheduling and route allocation. The code is difficult to maintain and lacks modularity. I need you to refactor this code using modern programming practices — improve readability, modularity, and performance while maintaining the same

functionality. Please explain what changes were made and why, and provide a sample input and output for testing." **CODE:**

```
7.1.py > ...
import heapq
import bisect
import logging
from dataclasses import dataclass, asdict
from typing import List, Tuple, Dict, Optional

logger = logging.getLogger("scheduler")
logging.basicConfig(level=logging.INFO, format="%(levelname)s: %(message)s")

@dataclass(frozen=True)
class Vehicle:
    id: str
    capacity: int
    available_from: int # epoch minutes or simple integer time unit

@dataclass(frozen=True)
class RouteRequest:
    id: str
    passengers: int
    start_time: int # integer time unit (e.g., minutes)
    duration: int # duration in same time unit

@dataclass
class Assignment:
    request_id: str
    vehicle_id: Optional[str]
    start_time: int
    end_time: Optional[int]
```

```

def schedule_routes(
    vehicles: List[Vehicle],
    requests: List[RouteRequest],
    turnaround: int = 10
) -> Tuple[List[Assignment], List[RouteRequest]]:
    """
    Greedy, efficient scheduler:
    - Processes requests ordered by start_time.
    - Maintains a heap of vehicles by next-available time.
    - Moves vehicles that are free into a capacity-sorted list for  $O(\log V)$  selection.
    - Returns (assignments, unassigned_requests).

    Time complexity:  $O(R \log V + V \log V)$  where  $R = \text{\#requests}$ ,  $V = \text{\#vehicles}$ .
    """
    # Defensive copies & sort
    requests_sorted = sorted(requests, key=lambda r: r.start_time)
    # Heap: (available_time, vehicle_index, Vehicle)
    vehicle_heap: List[Tuple[int, int, Vehicle]] = []
    for i, v in enumerate(vehicles):
        heapq.heappush(vehicle_heap, (v.available_from, i, v))

    # Available-by-capacity list sorted by capacity: list of tuples (capacity, avail.
    avail_by_capacity: List[Tuple[int, int, int, Vehicle]] = []

    assignments: List[Assignment] = []
    unassigned: List[RouteRequest] = []

    for req in requests_sorted:
        # Move vehicles that are free by req.start_time into avail_by_capacity
        while vehicle_heap and vehicle_heap[0][0] <= req.start_time:
            avail_time, vidx, veh = heapq.heappop(vehicle_heap)
            entry = (veh.capacity, avail_time, vidx, veh)

```

```

def schedule_routes(
    bisect.insort(avail_by_capacity, entry)

    # Find first vehicle with capacity >= req.passengers
    idx = bisect.bisect_left(avail_by_capacity, (req.passengers, -1, -1, None))
    if idx < len(avail_by_capacity):
        # assign that vehicle
        capacity, avail_time, vidx, veh = avail_by_capacity.pop(idx)
        start = req.start_time
        end = start + req.duration
        # vehicle next available after turnaround
        next_avail = end + turnaround
        heapq.heappush(vehicle_heap, (next_avail, vidx, veh))
        assignments.append(Assignment(request_id=req.id, vehicle_id=veh.id, start_
        logger.debug("Assigned %s -> %s (cap %d) [%d-%d]", req.id, veh.id, capac
    else:
        # No vehicle free at that start time; try to see if some vehicle becomes
        # Here we mark unassigned (could be extended to delay/start-later logic)
        assignments.append(Assignment(request_id=req.id, vehicle_id=None, start_
        unassigned.append(req)
        logger.debug("Unassigned %s (passengers=%d at %d)", req.id, req.passenge

    return assignments, unassigned

```

```

if __name__ == "__main__":
    # Sample fleet
    fleet = [
        Vehicle(id="v1", capacity=4, available_from=0),
        Vehicle(id="v2", capacity=8, available_from=0),
        Vehicle(id="v3", capacity=2, available_from=30), # becomes available later
    ]

    # Incoming route requests (time units = minutes)
    requests = [
        RouteRequest(id="R1", passengers=3, start_time=10, duration=40),
        RouteRequest(id="R2", passengers=2, start_time=15, duration=20),
        RouteRequest(id="R3", passengers=5, start_time=20, duration=30),
        RouteRequest(id="R4", passengers=1, start_time=35, duration=15),
        RouteRequest(id="R5", passengers=2, start_time=50, duration=20),
    ]

    assignments, unassigned = schedule_routes(fleet, requests, turnaround=10)

    print("Assignments:")
    for a in assignments:
        print(asdict(a))
    print("\nUnassigned requests:")
    for r in unassigned:
        print([asdict(r)])

```

OUTPUT:


```

ser s\varsh\OneDrive\Desktop\betch\2-1\wt\17.1.py
• Assignments:
{'request_id': 'R1', 'vehicle_id': 'V1', 'start_time': 10, 'end_time': 50}
{'request_id': 'R2', 'vehicle_id': 'V2', 'start_time': 15, 'end_time': 35}
{'request_id': 'R3', 'vehicle_id': None, 'start_time': 20, 'end_time': None}
{'request_id': 'R4', 'vehicle_id': 'V3', 'start_time': 35, 'end_time': 50}
me': 70}
me': 70}

Unassigned requests:
{'id': 'R3', 'passengers': 5, 'start_time': 20, 'duration': 30}
PS C:\Users\varsh\OneDrive\Desktop\betch\2-1\wt>

me': 70}

```

OBSERVATION:

1. The code uses clear dataclasses (Vehicle, RouteRequest, Assignment) and a single schedule_routes function, improving readability and testability.
2. A deterministic greedy algorithm with a min-heap and capacity-sorted list enables fast vehicle assignment, reducing unnecessary scans.
3. Heap and bisect.insort operations keep scheduling logarithmic, suitable for medium-scale fleets; scalable options like segment trees or capacity buckets can be added if needed.
4. Logging (debug/info) is included to trace scheduling decisions, and the purePython design with no global state makes the code easy to unit-test.